

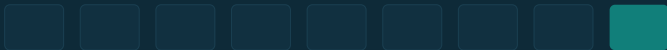


STREAM PROCESSING AVEC APACHE KAFKA

Producers & Consumers avancés

Clés & partitionnement · groupes & rebalance · commit des offsets

ESGI · 5^e année · Data Engineering · 2025–2026



SÉANCE 5

Objectifs

- **Router avec la clé**

Choisir une clé, comprendre `murmur2`, garantir l'ordre par clé.

- **Partager le travail dans un groupe**

Assignment partition → consommateur, parallélisme et ses limites.

- **Vivre un rebalance**

Déclencheurs, stratégies d'assignation, et le nouveau protocole KIP-848.

- **Maîtriser les offsets**

Commit auto vs manuel, garanties at-least-once / at-most-once.

SÉANCE 5

Au programme

01

Clés & partitionnement

Router, ordre par clé

02

Consumer groups & rebalance

Assignment, KIP-848

03

Commit & garanties

at-least / at-most-once

04

Atelier guidé

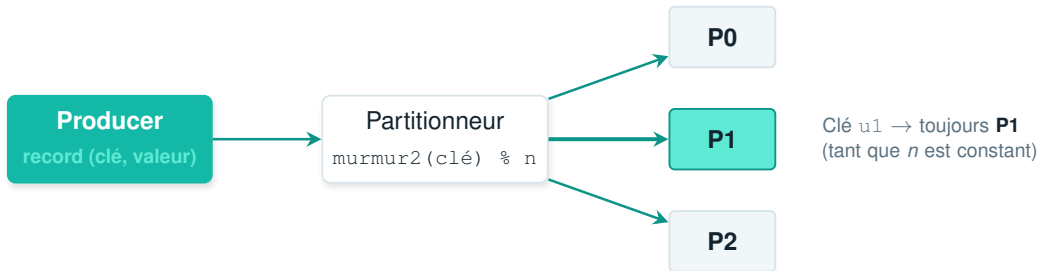
3 brokers, rebalance live

01

- PARTIE 1

Clés & partitionnement

Qui décide de la partition ?



- **Clé présente** : `partition = murmur2(clé) % nbPartitions` — déterministe, même clé → même partition.
- **Clé nulle** : partitionneur sticky-uniforme intégré (KIP-794) — réparti pour le débit, sans garantie d'ordre par clé.
- L'ordre n'est garanti que *dans une partition* — donc **par clé**, jamais à l'échelle du topic.

Bien choisir sa clé

■ Ordre

Garanti par partition, donc par clé. Pas de clé $\Rightarrow$ pas d'ordre métier.

■ Skew

Clé à faible cardinalité (région, statut) $\Rightarrow$ partitions chaudes, charge déséquilibrée.

■ Changer n

Modifier le nombre de partitions casse le mapping clé $\rightarrow$ partition. Préférer recréer un topic.

■ Compaction

Sur un topic compacté, la clé est l'identité : utiliser un ID immuable, jamais un email.

Voir la clé décider la partition

Produire avec / sans clé, puis lire la partition

```
# Produire AVEC clé, format "userId:message" (séparateur ':')
kafka-console-producer.sh --bootstrap-server localhost:29092 \
  --topic events --property parse.key=true --property key.separator=:
> u1:commande A      > u1:commande B      > u2:commande C
# Relire en affichant la partition ET la clé
kafka-console-consumer.sh --bootstrap-server localhost:29092 --topic events \
  --from-beginning --property print.partition=true --property print.key=true
```

■ Avec clé

Les u1 sur la **même partition** ⇒ ordre garanti.

■ Sans clé

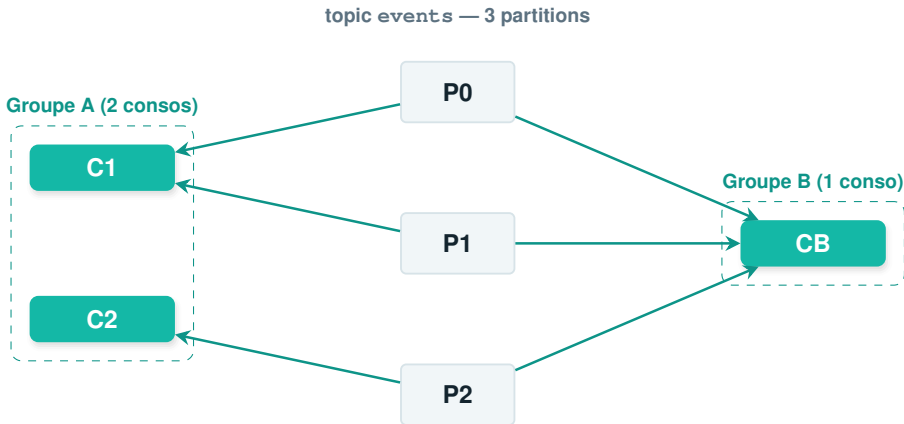
Sans `parse.key` : réparti, ordre par user perdu.

02

- PARTIE 2

Consumer groups & re-balance

Un groupe se partage les partitions



- Dans un groupe : **1 partition** → **au plus 1 consommateur** (l'ordre par partition est préservé).

Jusqu'où ça scale ?

■ Monter en débit

Ajouter des consos
augmente le débit

jusqu'à

`nbPartitions`.

■ Le plafond

Au-delà : consos oisifs.

Pour aller plus loin,
ajouter des partitions.

■ Plusieurs vues

N groupes = N lectures
indépendantes du même
flux (analytics, audit. . .).

Dimensionner le nombre de partitions *en amont*, selon le parallélisme visé à 12–18 mois —
c'est le levier de scaling d'un groupe, et il est coûteux à changer après coup.

Quand l'assignation est rejouée

■ Déclencheurs

Un conso rejoint, part, ou tombe (timeout) ;
ou des partitions sont ajoutées au topic.

■ Effet

Les partitions sont réassignées entre les
membres vivants du groupe.

- **Stratégies d'assignation** (protocole classique) : `range`, `round-robin`, `sticky`, `cooperative-sticky`.
- La variante *coopérative* évite de tout révoquer : on ne déplace que les partitions nécessaires.
- **Static membership** (`group.instance.id`) : un redémarrage bref ne déclenche pas de rebalance.

Le nouveau protocole de rebalance

- Piloté **côté serveur** (le *group coordinator*), incrémental et coopératif : plus de barrière de synchronisation globale, donc plus de « stop-the-world ».
- **GA en Kafka 4.0**, mais **opt-in côté client** : le défaut client reste `classic`.
- L'assignation s'appuie sur un assignor serveur (`UniformAssignor` par défaut).

Activer le protocole cote consommateur (`consumer.properties`)

```
# Opt-in (le default client reste "classic")
group.protocol=consumer
# Optionnel : forcer un assignor cote serveur
group.remote.assignor=uniform
```

Provoquer un rebalance en direct

Lancer plusieurs consos dans le meme group.id

```
# Terminaux 1, 2, 3 : meme group.id -> ils se partagent les 3 partitions  
mvn -q exec:java -Dexec.mainClass=fr.esgi.kafka.tp5.GroupConsumer
```

```
# Observer l'assignation membre -> partitions, et le LAG par partition  
kafka-consumer-groups.sh --bootstrap-server localhost:29092 \  
  --describe --group atelier-s5
```

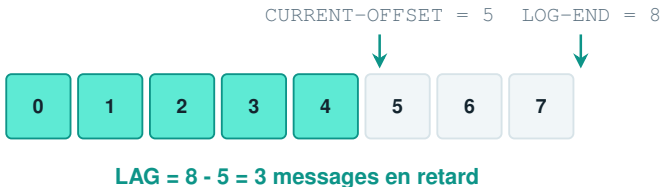
À observer : avec 1 conso → il détient P0,P1,P2. On lance C2 puis C3 ⇒ l'assignation se redistribue. On coupe un conso (Ctrl-C) ⇒ ses partitions repartent vers les autres.

03

- PARTIE 3

Commit des offsets & garanties

Où en est mon groupe ?



- L'offset committé = le **prochain** message à lire ; il est stocké dans le topic interne `__consumer_offsets`, clé par (group, topic, partition).
- **LAG** = LOG-END-OFFSET - CURRENT-OFFSET : le retard du groupe sur la production.

Quand committer change tout

Configuration	Garantie / risque
<code>enable.auto.commit=true</code>	Commit périodique automatique : simple, mais découplé du traitement réel.
Manuel, après traitement	at-least-once : un crash avant commit $\Rightarrow$ retraitement (doublons possibles).
Manuel, avant traitement	at-most-once : un crash après commit $\Rightarrow$ message perdu.
Idempotence + transactions	exactly-once (EOS) $\rightarrow$ détaillé en Séance 6.

En pratique : `enable.auto.commit=false + commitSync()` *après* traitement = at-least-once.

Passer le consumer en commit manuel

GroupConsumer.java : boucle poll en commit manuel (at-least-once)

```
props.put(ConsumerConfig.GROUP_ID_CONFIG, "atelier-s5");
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false"); // manuel
props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

consumer.subscribe(List.of("events"));
while (running) {
    var records = consumer.poll(Duration.ofMillis(500));
    for (ConsumerRecord<String, String> r : records) {
        handle(r); // traitement idempotent
    }
    consumer.commitSync(); // commit APRES traitement (at-least-once)
}
```

Test : couper le consumer *avant* `commitSync()` $\Rightarrow$ au redémarrage, les mêmes messages sont rejoués.

04

• PARTIE 4

Atelier guidé

Le cluster et le topic

Demarrer 3 brokers KRaft et creer le topic

```
# Cluster 3 brokers (mode KRaft) fourni dans le kit
docker compose up -d

# Topic de 1 atelier : 3 partitions, replication 3
kafka-topics.sh --bootstrap-server localhost:29092 \
  --create --topic events --partitions 3 --replication-factor 3

# Verifier l etat (leaders, ISR, replicas)
kafka-topics.sh --bootstrap-server localhost:29092 --describe --topic events
```

■ Kit fourni

`docker-compose.yml`, projet Maven

- ESGI : Stream Processing avec Apache Kafka (starter + corrigé), README pas-à-pas.

■ Bootstrap

Depuis l'hôte : `localhost:29092`. Un seul broker suffit pour amorcer.

Le parcours guidé

- 1 Compléter le **producteur** (TODO) : `clé = userId`, envoyer 30 messages répartis sur quelques utilisateurs.
- 2 Lancer **1 consumer** : vérifier qu'il détient P0, P1, P2 via `-describe`.
- 3 Lancer un **2^e puis un 3^e consumer** (même `group.id`) → observer le **rebalance**.
- 4 Couper un consumer → observer le rebalance inverse et le LAG transitoire.
- 5 Compléter le passage en **commit manuel** (`commitSync`) et tester le rejou.

Lire la sortie de `-describe`

```
kafka-consumer-groups -describe -group atelier-s5 (extrait)
```

TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID
events	0	120	125	5	consumer-1
events	1	98	98	0	consumer-1
events	2	77	90	13	consumer-2

■ Assignment

CONSUMER-ID par partition : ici C1 détient P0+P1, C2 détient P2.

■ Santé

LAG > 0 et qui monte $\Rightarrow$ le groupe ne suit pas le débit de production.

Pour aller plus loin

Objectif — garantir l'ordre *par utilisateur* tout en restant **at-least-once**, et mesurer le comportement du LAG pendant un rebalance.

■ 1. Ordre

Clé = `userId` : prouver via `-describe` / la sortie que l'ordre tient par utilisateur.

■ 2. LAG

Produire en continu, déclencher un rebalance, tracer le LAG avant / pendant / après.

■ 3. Bonus

Static membership + protocole `consumer` (KIP-848) : comparer l'impact.

SÉANCE 5

À retenir

- ✓ La clé route via `murmur2 % n` et garantit l'ordre *par clé* (jamais par topic).
- ✓ Dans un groupe : 1 partition $\rightarrow$ 1 conso ; le débit d'un groupe plafonne au nombre de partitions.
- ✓ Le rebalance réassigne les partitions ; **KIP-848** le rend incrémental (opt-in `group.protocol=consumer`).
- ✓ Commit **après** traitement = at-least-once ; **avant** = at-most-once.
- ✓ `kafka-consumer-groups -describe` : l'outil n°1 pour l'assignation et le LAG.



LA SUITE — SÉANCE 6

Sémantiques de livraison & exactly-once

Idempotence, transactions, et l'ouverture vers les share groups / Queues for Kafka (conso > partitions).